

Block Buddies: A Dynamic, Nested Curriculum Matrix for Peer-to-Peer Computer Science Mentorship

Executive Summary: The Pedagogical and Technical Architecture

The "Block Buddies" program represents a synthesis of constructivist learning theory, specifically Lev Vygotsky's Zone of Proximal Development (ZPD), and rigorous technical skill acquisition in Computer Science (CS). This comprehensive operational framework outlines a Dynamic, Nested Curriculum Matrix designed to empower Teen Mentors (ages 14-17) to guide Junior Mentees (ages 8-13) through the complexities of Java-based Minecraft modding.

Unlike static lesson plans that prescribe a linear path, this matrix functions as a complex "Decision Tree." It acknowledges that a rigid curriculum fails when facing the variable baselines of modern digital literacy. Recent educational research and anecdotal evidence from Computer Science educators indicate a growing "file system deficit" among Generation Alpha—students who are highly proficient with touch interfaces and app-based ecosystems but lack a mental model of hierarchical file structures, directories, or local storage management.¹ This curriculum explicitly addresses these deficits by treating the Minecraft directory (.minecraft) not merely as a game folder, but as a primary text for teaching Operating System fundamentals, file hierarchy, and system administration.

The objective is twofold: to build technical competency in the Mentee—covering CSTA standards, logic, and systems thinking—and to develop instructional empathy and leadership in the Mentor. The curriculum maps strictly to CSTA K-12 Computer Science Standards, ensuring that what feels like play is, structurally, an academic progression from basic computing systems to complex algorithmic thinking.³ This document serves as the authoritative manual for Mentors, differentiating instruction based on Tech Fluency, Archetype (Motivation), and Age Group.

Section 1: The Diagnostic Protocol ("The Sorting Hat")

The efficacy of the "Block Buddies" program relies entirely on accurate initial assessment. We cannot assume prior knowledge based on age or self-reported "gamer" status. A 12-year-old "Level 1 Gamer" may be unable to locate a downloaded file, while a 9-year-old "Level 2 Tinkerer" might already intuit boolean logic through Redstone play. The Diagnostic Protocol acts as a "Sorting Hat," using practical, observed tasks to categorize the Mentee into a specific profile that dictates their starting branch in the curriculum matrix.

1.1 The Theoretical Basis of the Diagnostic

The diagnostic is designed to bypass the "Dunning-Kruger" effect where students might over- or underestimate their skills. Instead of asking "Do you know how to use files?", the Mentor observes the *method* of execution. The focus is on the "Process," not just the "Product." We are looking for the existence of a mental model of the computer's file system.¹ Does the student see the computer as a magic box of apps (iPad Native) or as a structured hierarchy of containers (Tinkerer)?

1.2 The 5-Minute Practical Test Script

Context for Mentor: This is not a test. It is framed as a "Setup Game" or "Scavenger Hunt" to prepare the computer for modding. The Mentor must observe the *speed* and *method* of execution.

Task 1: The "Screenshot Scavenger Hunt" (File Systems)

Objective: Determine if the student understands file paths, extensions, and directory creation.

Mentor Script: "Okay, first challenge to get our badge. I want you to take a screenshot of your current desktop background. Save that picture as secret_base.png. Then, I need you to create a new folder on your desktop called BlockBuddies and move that picture inside it."

Observation Metrics:

- **The iPad Native (Level 0):** Takes a photo of the screen with a phone. Searches the Start Menu for "folder." Leaves the file in "Downloads" or "Recent Files" and cannot move it. Struggles with right-click context menus.
- **The Gamer (Level 1):** Uses the "Print Screen" button but pastes it into Word or Paint. Can save it, but relies on the default location. Drag-and-drop is hesitant.
- **The Tinkerer (Level 2):** Uses Win+Shift+S or Cmd+Shift+4. Right-clicks desktop -> New Folder immediately. Renames the file during the save process.

Task 2: The "Mod Check" (Navigation)

Objective: Determine familiarity with the Minecraft application data structure.

Mentor Script: "Have you ever played with mods before? Show me where your Minecraft mods folder is hiding on this computer."

Observation Metrics:

- **The iPad Native (Level 0):** "What is a folder?" or "I usually play on my iPad." Looks at the desktop for a game icon.
- **The Gamer (Level 1):** Opens the Minecraft Launcher and looks through the game menus. Knows it exists but relies on a mod-loader app (like CurseForge) to manage it for them.
- **The Tinkerer (Level 2):** Opens File Explorer. Navigates to %appdata%/.minecraft/mods (Windows) or ~/Library/Application Support/minecraft/mods (Mac) without hesitation.

Task 3: The "Motivation Probe" (Archetype Assignment)

Objective: Align the curriculum with the student's intrinsic motivation to ensure sustained engagement.

Mentor Script: "If you were the Lead Developer of Minecraft and could add one thing right now, what would it be? A giant castle with furniture, a robot that mines diamonds for you, or a ban hammer to stop your brother from messing up your world?"

Archetype Assignment:

- *Castle/Furniture*: **The Visualist (Builder)**.
- *Robot/Mining*: **The Mechanist (Engineer)**.
- *Ban Hammer/Rules*: **The Operator (Admin)**.

1.3 The Decision Tree Matrix

Based on the observations, the Mentor assigns a Profile Code. This code is the key to the rest of the document.

Variable	Profile Name	Indicators & Behaviors	Instructional Strategy (The "How")
Tech Fluency 0	The iPad Native	Cannot distinguish between internet search and local files. Struggles with right-click/left-click context. No concept of "save location."	Heavy Scaffolding. 80% of time spent on OS navigation. Use analogies (House = Drive, Room = Folder). "Win Buttons" are frequent.
Tech Fluency 1	The Gamer	High dexterity in-game. Can install software via "Wizards" but fears "breaking" the computer. Relies on GUI tools exclusively.	Bridge Building. Leverage game confidence to teach backend logic. "To fix the lag, we must edit the file." Transition from consumer to creator.
Tech Fluency 2	The Tinkerer	Has broken the game before. Familiar with	Challenge Mode. Focus on <i>why</i> things work.

		%appdata%. May have copied/pasted code but doesn't understand syntax.	Introduce syntax discipline and debugging workflows. Allow them to fail safely.
Archetype A	The Visualist	Focus on aesthetics, colors, lighting, textures.	Entry Point: Resource Packs (JSON editing), Shaders (Rendering Pipeline).
Archetype B	The Mechanist	Focus on automation, farms, efficiency, complex systems.	Entry Point: Redstone (Logic Gates), Create Mod (Physics/Kinetic Energy).
Archetype C	The Operator	Focus on control, rules, multiplayer dynamics, commands.	Entry Point: Server Properties, Permissions (Boolean Logic), Command Blocks.

Section 2: The Core Trunk (Universal Skills)

Before any specialized modding occurs, all students must master the **Core Trunk**. These are the "Hidden Computer Science" lessons. The student thinks they are preparing to mod; the Mentor knows they are learning Systems Administration, File Hierarchy Standards, and Data Safety. This section maps directly to **CSTA Standards 1B-AP-15 (Test and Debug)**, **2-AP-19 (Document programs)**, and **1B-CS-02 (File/System Organization)**.⁴

2.1 Module 1: The "Digital Backpack" (File Systems & Navigation)

The Problem: As identified in recent pedagogical studies and reports from CS professors, Generation Alpha often views the computer as a "magic box" where files disappear into a "cloud" rather than a hierarchical storage device. They may lack the vocabulary for "directories," "extensions," or "paths".¹ This "File System Deficit" is the primary barrier to technical literacy.

The Objective: The Mentee will navigate to the Minecraft directory using absolute paths,

create a dedicated workspace, and identify the difference between a file extension (.jar, .json) and a file name.

Nested Differentiation: File Navigation

Tech Fluency	Instructional Approach (The "How")	Mentor Script (Key Interaction)	CSTA Alignment
Level 0 (Junior)	The House Analogy. The Hard Drive is the House. Users are the People. Desktop is the Living Room. We need to go to the "Basement" (AppData) where the pipes are.	"We are going to find the secret brain of Minecraft. It's hiding in a folder called AppData. Think of it like a secret level in a game. We have to open the right doors to find it."	1A-CS-01: Select and operate computing devices.
Level 1 (Senior)	The Direct Path. Focus on efficiency. Introduce %appdata% as a variable—a shortcut the computer uses to find your specific room in the house.	"Stop clicking folders one by one. Hit Win+R. Type %appdata%. This is a 'variable' that tells Windows 'Find the current user.' It's faster and pro-level."	1B-CS-02: Model how computer hardware and software work together.
Level 2 (Any)	The Command Line. Skip the mouse. Introduce the terminal/command prompt for navigation (cd, dir, ls).	"Real engineers don't drag-and-drop. Open PowerShell. Navigate to the .minecraft folder using only the keyboard. Type cd to change directory."	2-CS-02: Design projects that combine hardware and software components.

The Hidden CS Lesson: Hierarchical Data Structures.

The Mentee learns that data is stored in a tree structure (Root -> Branch -> Leaf). This is the foundational concept for understanding decision trees, databases, and DOM manipulation later in life.

2.2 Module 2: The "Time Machine" (Backups & Version Control)

The Problem: Modding inevitably breaks the game. Without a safety net, a crash leads to data loss, tears, and program abandonment.

The Objective: Mentee will create a "Clean Backup" before applying any changes, effectively learning the concept of a "Commit" in version control systems like Git.

Protocol: The "Zip & Label" Strategy

1. **Stop:** Before installing a mod loader (Forge/Fabric).
2. **Select:** Highlight the saves and options.txt files.
3. **Compress:** Create a .zip archive.
4. **Label:** Rename the zip using ISO 8601 date format or a semantic version (e.g., WorldBackup_2023-10-27_v1).

Differentiation Strategy:

- **For Juniors (The Save Point):** "We are making a 'Save Point,' just like in a video game before a boss fight. If the mod crashes the game (the boss defeats us), we can just respawn right here."
- **For Seniors (The Git Commit):** "This is Version Control. In the software industry, we call this a 'Commit.' If you push bad code that breaks the build, you 'revert' to this snapshot. Naming it clearly is crucial so you know which version works."

2.3 Module 3: Digital Hygiene & Safety (The Mod Supply Chain)

The Problem: The internet is filled with "fake download buttons," adware, and malware disguised as Minecraft mods. Young users often lack the discernment to identify safe sources.

The Objective: Mentee will identify legitimate sources (CurseForge, Modrinth) and verify file integrity by checking file extensions and author reputation.⁶

Decision Tree: Is this file safe?

- **IF** URL contains minecraft-free-mods.net or similar generic names -> **STOP**. Explain Adware/Malware risk.
- **IF** File extension is .exe or .msi (when expecting a Java .jar) -> **STOP**. Explain Executable risks.
- **IF** Source is CurseForge or Modrinth AND Author has >10,000 downloads -> **PROCEED**.
- **IF** The "Download" button is flashing or is a different color than the site theme -> **STOP**. It is likely an ad.

Mentor Script: "We are the security guards of our computer. We only let verified trucks (files)

through the gate. If a truck looks suspicious or comes from a strange place, we turn it away."

Section 3: The Branching Paths (Specializations)

Once the Core Trunk is established (usually by Session 3), the Mentor pivots to the student's **Archetype**. This is where **Differentiated Instruction** becomes the primary driver. Each track teaches high-level CS concepts disguised as gameplay customization.

Track A: The Visualist (The Builder)

Target Profile: Loves aesthetic builds, skins, and "pretty" graphics.

Hidden CS Concepts: Rendering Pipelines, RGB Color Models, JSON Syntax, Coordinate Systems.

Module A1: Resource Packs & JSON (The "Code of Paint")

Objective: Change the color of a diamond sword or the texture of dirt using a custom Resource Pack.

Technical Context: Minecraft textures use JSON files to map 2D images to 3D models. The file structure must be exact: `assets/minecraft/textures/item.7`

Step-by-Step Scaffolding:

1. **Unzipping the Assets:** Extract the vanilla .jar file to reveal the assets folder. This demystifies the game—it's just a collection of files.
2. **The Hierarchy:** The Mentee must replicate the exact folder structure. *Lesson: Strict path dependency.*
3. **The pack.mcmeta File:** This is the metadata file. It must contain specific JSON syntax including the `pack_format` integer (which changes by version, e.g., format 15 for 1.20).⁸
 - o **Common Friction Point:** Missing commas or unclosed brackets `{}`.
 - o **Mentor Script:** "The computer is very literal. It doesn't know what you *meant* to do; it only knows what you typed. A missing comma breaks the whole logic. This is called 'Syntax'."

Differentiation (JSON Editing):

- **Level 0/Junior:** Use a JSON validator tool online. Copy-paste a template and only change the description text.
- **Level 2/Senior:** Write the JSON from scratch in VS Code or Notepad++. Explain "Key-Value Pairs" and data types (Integer vs String).

Module A2: Shaders & The GPU (The "Light Physics")

Objective: Install Shaders (OptiFine/Iris) to make water reflect and trees sway.

Hidden CS Concept: The Rendering Pipeline (Vertex vs. Fragment Shaders).⁹

The Explanation (Deep Insight):

Shaders are not just "mods"; they are programs that run on the Graphics Card (GPU), not the Central Processing Unit (CPU).

- **Concept for Seniors:** Explain the difference between the **Vertex Shader** (manipulates the geometry/shape, e.g., waving grass coordinates) and the **Fragment Shader** (manipulates the pixel color, e.g., lighting, shadows, reflections).⁹
- **Analogy for Juniors:** "The CPU is the architect drawing the house. The GPU is the painter coloring it in. Shaders tell the painter to use glitter paint instead of matte paint."

Troubleshooting Scenario (High Value):

- *Problem:* FPS drops to 10 after installing shaders.
- *Solution:* Discuss hardware limitations. "Your GPU has a budget. Shadows cost \$100. Reflections cost \$200. You only have \$150. What do we cut?" (Introduction to Resource Management and Optimization).

Track B: The Mechanist (The Engineer)

Target Profile: Loves Redstone, automation, and complex machinery.

Hidden CS Concepts: Logic Gates, Boolean Algebra, Energy Systems, Calculus (Rates of Change), Signal Processing.

Module B1: Redstone Logic (The "Physical Circuit")

Objective: Build a secret door using a passcode (AND Gate) or a toggle switch (T-Flip Flop).

Technical Context: Redstone is a Turing-complete language capable of representing binary logic.¹⁰

The Curriculum Map:

1. The NOT Gate (Inverter):

- *Build:* Redstone torch on a block. Input ON -> Output OFF.
- *CS Concept:* Boolean NOT (!). `if (!button_pressed)`.

2. The OR Gate:

- *Build:* Two levers connected to one door. Either lever opens it.
- *CS Concept:* Boolean OR (||). `if (leverA |`

`| leverB)`. 3. **The AND Gate:** * *Build:* Three torches and dust. Both levers must be down to turn the output torch on. * *CS Concept:* Boolean AND (&&). `if (leverA && leverB)` `.

* Mentor Script: "This is how passwords work. You need the Username AND the Password to get in. Not just one."¹¹

Module B2: The Create Mod (The "Physics Engine")

Objective: Build an automated wheat farm or a steam engine.

Hidden CS Concept: Kinetic Energy, Rotational Speed vs. Torque (Stress Units), Gear Ratios.

Key Concept: Stress Units (SU) vs. RPM

- **The Physics:** The "Create" mod separates speed (RPM) from strength (Stress).
- **The Math:** Total Stress Capacity = Generator Strength × RPM.¹³
- **The Lesson:** If a machine overstresses, you must either *increase capacity* (add more engines) or *decrease consumption* (gear down/slow down). This is an introduction to

System Load Balancing and **Mechanical Engineering** principles.

Differentiation:

- **Junior:** "The machine is too heavy. We need to make it spin slower to have more muscle." (Bicycle gear analogy - low gear for hills).
- **Senior:** Calculate the exact SU requirement. "The mechanical press uses 2048 SU. Our water wheel produces 512 SU. How many wheels do we need? Or can we gear it down by a ratio of 4:1?" (Algebra and Ratio application).

Track C: The Operator (The Admin)

Target Profile: Wants to own a server, ban griefers, and control the world rules.

Hidden CS Concepts: Networking (IPs/Ports), Permissions (ACLs), Linux/Server Configuration, Text-based Configuration.

Module C1: The Server Properties File (The "Constitution")

Objective: Set up a local host server and modify server.properties.

Technical Context: Configuring a daemon process via a flat-file database.¹⁵

Key Variables to Edit:

- gamemode=survival (State definition)
- pvp=true/false (Boolean flag)
- server-port=25565 (Networking Endpoint)

The Hidden Lesson: Networking Layer

- **The Analogy:** "The IP Address is your house address on the street. The Port (25565) is the specific door to your room. If you don't unlock the door (Port Forwarding), the mailman (data) hits a wall."¹⁷
- **Safety Note:** For Level 0/Junior students, use "LAN" or Hamachi/Tailscale/Ngrok rather than real router port forwarding to avoid exposing home networks to the public internet.

Module C2: Permissions & Logic (LuckPerms)

Objective: Create ranks (Admin, Moderator, Guest) with different abilities using the LuckPerms plugin.

Hidden CS Concept: Access Control Lists (ACLs) and Inheritance.¹⁸

Implementation:

- **Concept: Inheritance.** If the "Admin" group inherits from "Moderator," it gets all Mod permissions plus new ones. This teaches Object-Oriented Programming (OOP) concepts like class inheritance.
- **Boolean Logic in Perms:** minecraft.command.ban = true vs minecraft.command.ban = false.
- **Mentor Script:** "We don't give people power they don't need. This is the 'Principle of Least Privilege.' Why shouldn't a Guest have the ban command? If a Guest account gets

hacked, we don't want the hacker to burn the world."

Section 4: The Troubleshooting Meta-Skill (The Friction Points)

This is the most critical section of the "Block Buddies" program. In standard education, the teacher often removes friction to facilitate smooth learning. In this program, **we engineer friction**. The curriculum intentionally includes moments where things *will* break (e.g., incompatible mods, syntax errors), forcing the Mentee to engage in the "Troubleshooting Loop."

CSTA Standard 1B-AP-15: Test and debug (identify and fix errors) a program or algorithm to ensure it runs as intended.⁴

4.1 The Troubleshooting Algorithm (Decision Tree)

When an error occurs (e.g., The game crashes on launch), the Mentor must NOT fix it immediately. They must guide the Mentee through this cognitive tree:

1. **Read the Logs:** "Don't just click close. What does the text say? Computers tell us what hurts, we just have to listen."
2. **Identify the Keyword:** Look for specific error classes: NullPointerException, MissingDependency, or Exit Code.
3. **Isolate the Variable:** "We installed three mods. Which one is the traitor? How do we find out?" (Teach the Binary Search method: Remove half the mods, test, repeat).
4. **Google-Fu:** Teach the art of the search query.
 - *Bad Search:* "Minecraft broken help."
 - *Good Search:* "Minecraft 1.20.1 Exit Code 1 Create Mod crash."

4.2 Common "Friction Points" & Solutions Matrix

Error / Symptom	The Technical Cause	The Pedagogical Opportunity (Mentor Script)	The "Win Button" (Junior Strategy)	The Challenge Mode (Senior Strategy)
Exit Code 1	General Java Error / Mod Mismatch / Driver Issue. Often caused	"Code 1 is a generic 'I give up' from Java. We need to play detective.	"Let's delete the mods folder and put them back one by one until it	"Open the latest.log file in Notepad. Search for 'Exception' or

	by version incompatibility. ¹⁹	Did we mix Forge and Fabric mods? Those are two different languages."	breaks." (Brute force).	'Error'. Tell me what the first error line says." (Log Analysis).
"Missing Dependency"	A mod requires a library (e.g., JEI needs a config lib).	"This mod is like a car, but we forgot to buy the wheels. Read the error—what 'part' is it asking for?"	Mentor provides the direct link to the missing file immediately.	Mentee must copy the filename from the error message, find it on CurseForge/M odrinth themselves, and verify the version match.
Purple/Black Textures	Missing texture file or broken path in JSON. ⁷	"The game can't find the picture you promised it, so it uses this ugly purple as a warning sign. Check your file names."	"Let's check the spelling of the file name together. Look, 'diamond' vs 'daimond'."	"Check your JSON file structure. Did you use an uppercase letter? Minecraft assets must be lowercase only. Linux file systems are case-sensitive." "
Redstone Doesn't Power	Signal decay (Redstone only travels 15 blocks).	"The electricity ran out of juice. It's too far. How do we boost the signal?"	"Here is a Repeater block. Place it here."	"I'm not telling you. Build a test line and count how many blocks the power travels before it dies. Then

				design a relay station."
"Out of Memory"	Java Heap Space exhaustion.	"Your backpack is full. The game needs more RAM to hold all these mods."	Mentor shows where the RAM slider is in the launcher settings.	Mentee must calculate available system RAM and modify the JVM arguments (-Xmx4G) manually.

4.3 Scaffolding Strategies (Vygotsky in Action)

The Mentor must dynamically adjust their support level based on the Mentee's frustration level. This is the **ZPD Calibration**.

- **Level 1: Modelling (I do, you watch).**
 - *Trigger:* Mentee is completely lost or Tech Fluency 0.
 - *Action:* Mentor shares screen, narrating every click. "I am clicking 'Save As', navigating to Desktop, typing the name..."
- **Level 2: Co-Construction (We do together).**
 - *Trigger:* Mentee understands the goal but forgets the syntax or step.
 - *Action:* "Okay, you have the file. Where did we say the mods folder lives? It starts with a percent sign..."
- **Level 3: Fading (You do, I watch).**
 - *Trigger:* Mentee has done this task at least once successfully.
 - *Action:* "I want you to install this new mod. I'm going to turn off my camera/share. Tell me when it's done."

Section 5: The Mentor's Toolkit - Advanced Pedagogy

This section provides the Teen Mentor with the soft skills required to be an effective teacher. It emphasizes that the goal is not to show off how much the Mentor knows, but to facilitate the Mentee's discovery.

5.1 The Art of the Question (Socratic Method)

Instead of giving answers, Mentors should use "Guiding Questions" to lead Mentees to the solution.²¹

Bad Mentor Response	Good Mentor Response	Purpose
---------------------	----------------------	---------

	(Socratic)	
"You put the file in the wrong folder."	"Where does the game look for mods? Does it look on the Desktop or somewhere else?"	Forces recall of the file path concept.
"Here, let me fix that code for you."	"The game is giving a syntax error. It says 'Unexpected Token'. What does a JSON list usually end with?"	Encourages syntax analysis.
"That won't work."	"What do you think will happen if we run it like this? Let's try it and see."	Encourages hypothesis testing and scientific method.

5.2 Managing Frustration (Emotional Regulation)

Coding is frustrating. Kids will rage-quit. The Mentor's job is to normalize failure.

- **The "Rubber Duck" Method:** Teach the Mentee to explain their problem to an inanimate object (or the Mentor) line by line. Often, explaining the problem reveals the solution.
- **The 5-Minute Rule:** For Seniors, allow them to struggle with a problem for 5 minutes before intervening. This builds resilience. For Juniors, intervene after 30 seconds to prevent disengagement.²³

Conclusion: From Gamer to Developer

The "Block Buddies" Curriculum Matrix is not merely about playing Minecraft. It is a Trojan Horse for computer literacy. By the end of the program, a "Level 0 iPad Native" who successfully installs a shader pack has learned fundamental OS navigation. A "Level 2 Tinkerer" who automates a farm has learned systems engineering.

The Teen Mentor, in turn, masters the soft skills of technical leadership: empathy, communication, and the ability to explain complex abstractions in concrete terms. This recursive learning model—where the teacher learns by teaching—ensures that the program produces not just better gamers, but competent digital citizens and future engineers. The matrix provided here is the map; the journey is defined by the collaborative problem-solving between Mentor and Mentee.

Works cited

1. How can you judge who's tech illiterate vs tech literate, especially for gen z and alpha? - Reddit, accessed January 12, 2026, https://www.reddit.com/r/AskTechnology/comments/1os7zxw/how_can_you_judge_whos_tech_illiterate_vs_tech/
2. Students don't know what folder and folders are and lack of basic file management - Reddit, accessed January 12, 2026, https://www.reddit.com/r/Professors/comments/tskkah/students_dont_know_what_folder_and_folders_are/
3. CSTA K-12 Computer Science Standards, Revised 2017 - New Mexico Public Education Department, accessed January 12, 2026, <https://web.ped.nm.gov/wp-content/uploads/2025/01/CSTA-K-12-Computer-Science-Standards-Revised-2017-2.pdf>
4. K-12 Computer Science Standards - Computer Science Teachers ..., accessed January 12, 2026, <https://csteachers.org/wp-content/uploads/2025/03/csta-k-12-computer-science-standards-revised.pdf>
5. Computer Science K-12 Standards - OSPI, accessed January 12, 2026, <https://ospi.k12.wa.us/sites/default/files/2022-12/cs-standards.pdf>
6. Introduction to Computing - Let's Make a Redstone Computer #1 - YouTube, accessed January 12, 2026, <https://www.youtube.com/watch?v=osFa7nwHHz4>
7. Minecraft - Folder Structure of a Resource Pack (Guide) - YouTube, accessed January 12, 2026, <https://www.youtube.com/watch?v=CIFdgbhHS8E>
8. How To Make A Texture Pack In Minecraft 1.21 - Resource Pack Tutorial - YouTube, accessed January 12, 2026, https://www.youtube.com/watch?v=J6fdkG_dilQ
9. ELI5 Shaders. What They Are, How They Work With Code, Why Are They Used, etc. - Reddit, accessed January 12, 2026, https://www.reddit.com/r/gamedev/comments/yh7pu7/eli5_shaders_what_they_are_how_they_work_with/
10. Computer Architecture Explained With MINECRAFT - YouTube, accessed January 12, 2026, https://www.youtube.com/watch?v=dV_lf1kyV9M
11. Beginners Logic Gates | Minecraft Education, accessed January 12, 2026, <https://education.minecraft.net/en-us/lessons/beginners-logic-gates>
12. How to Learn Logical Redstone [Full Guide] - YouTube, accessed January 12, 2026, <https://www.youtube.com/watch?v=u7CV7QBbA0I>
13. I need help understanding stress and speed : r/CreateMod - Reddit, accessed January 12, 2026, https://www.reddit.com/r/CreateMod/comments/lk67tg/i_need_help_understanding_stress_and_speed/
14. Minecraft Create Mod: A Beginner's Guide to Stress Units - YouTube, accessed January 12, 2026, <https://www.youtube.com/watch?v=H51XmzujQHl>
15. Server.properties File: An In-Depth Breakdown - Cybrancee, accessed January 12, 2026, <https://cybrancee.com/blog/server-properties-file-an-in-depth-breakdown/>

16. How to configure Minecraft server properties file - Hostinger, accessed January 12, 2026, <https://www.hostinger.com/tutorials/minecraft-server-properties>
17. Understanding IP Address and Port with a House Analogy: Layman's Guide - YouTube, accessed January 12, 2026, <https://www.youtube.com/watch?v=aOK6XPqTtUU>
18. How to Configure LuckPerms on Your Server - Knowledgebase - Shockbyte, accessed January 12, 2026, <https://shockbyte.com/billing/knowledgebase/249>
19. (Modpack Making) Minecraft Exit Code: 1 [SOLVED] + [How To Solve] : r/CurseForge, accessed January 12, 2026, https://www.reddit.com/r/CurseForge/comments/1bl7obj/modpack_making_minecraft_exit_code_1_solved_how/
20. How To Fix Exit Code 1 in Minecraft - YouTube, accessed January 12, 2026, <https://www.youtube.com/watch?v=H3rpVvzP90>
21. ChatGPT 4o Leetcode Coding Mentor that forces the student to think : r/ChatGPTPromptGenius - Reddit, accessed January 12, 2026, https://www.reddit.com/r/ChatGPTPromptGenius/comments/1fwqg01/chatgpt_4o_leetcode_coding_mentor_that_forces_the/
22. How to be great at asking coding questions | by Gordon Zhu - Medium, accessed January 12, 2026, https://medium.com/@gordon_zhu/how-to-be-great-at-asking-questions-e37be04d0603
23. 7 ways to use ZPD and scaffolding to challenge and support students - NWEA, accessed January 12, 2026, <https://www.nwea.org/blog/2025/7-ways-to-use-zpd-and-scaffolding-to-challenge-and-support-students/>